

Cohesify



Session 2024 - 2028

Submitted by:

Muhammad Umair 2024-CS-192

Supervised by:

Ma'am Rabeeya Saleem

Course:

CSC200 Data Structures and Algorithms

Department of Computer Science,
University of Engineering and Technology
Lahore, Pakistan

Contents

Cohesify Overview	4
Data Structures Used	5
Implementation Details	6
Application Outline.....	6
Data Structures	6
Arrays.....	6
Stacks	6
Queue	7
Linked List.....	7
Sorting Algorithms.....	7
Database.....	7
System Flow Diagram.....	8
Use Case: Posting and Image	9
Scope:	9
Level:.....	9
Primary Actor:	9
Stakeholders and Interests:.....	9
Preconditions:.....	9
Postconditions:	9
Main Success Scenario (Basic Workflow):	9
Extensions (Alternate Paths):	10
Frequency of Occurrence:.....	11
Applications Screenshots.....	12
1. Create Account.....	12
2. Login	13
3. Home	14
4. Friends	15
5. Profile.....	16
6. Search.....	18
7. Comments.....	19
Performance Analysis.....	20

List of Figures

Figure 1: Create account screen	12
Figure 2: Login screen	13
Figure 3: Home screen	14
Figure 4: Friends screen	15
Figure 5: Profile screen (a)	16
Figure 6: Profile screen (b)	17
Figure 7: Search screen	18
Figure 8: Comments screen	19

Cohesify Overview

Cohesify is a social media web application that allows users to interact through posts. It allows users to post pictures and videos with their friends and users, and interact with posts through likes and comments. It also provides sorting, filtering, and searching features on posts and people through a consistent UI. It is written in Django, utilizing Python for the backend; HTML, CSS, and JavaScript for the frontend; and SQLite as its database (Django ORM).

Data Structures Used

Like most real-world social media applications, Cohesify uses data structures and database queries to manage its features. Below is a detailed description of the data structures used.

Data Structure	Usage
Array	Sorting, filtering, and searching for posts and users.
Stack	Undo operation on the deletion of posts, and undo and redo for liking/unliking.
Queue	Implemented through a linked list for storing posts.
Linked List	Stores the currently signed-in user's friends' list.

Implementation Details

Application Outline

Cohesify has three main screens:

1. Home
2. Friends
3. Profile

Along with the above-mentioned screens, it offers:

1. Searching for users and posts
2. Filtering on users and posts

On every post, the following features are available:

1. Liking
2. Commenting

In order to keep track of the user's activity, and to allow backtracking, the following features are provided:

1. Undo and redo feature on the like/unlike action on posts
2. Undo feature on the deletion of posts

Data Structures

Arrays

Arrays are used for the following purposes.

- **Sorting** posts in **trending** order.
- **Sorting** posts by in terms of **latest** or **oldest**.
- **Filtering images**, and **videos**.
- **Sorting** people by name in **ascending** or **descending** order.
- Storing database query results.
- Handling the like icon on posts: If a post is liked by the currently logged-in user, the icon appears as pink filled heart. Otherwise, it appears as a hollow heart. This is done by querying the IDs of all the posts the user has liked, and checking whether the ID of the current post is in that array.
- Loading comments.

Stacks

The usage of stack is elaborated below.

- **Undo** and **redo** a like/unlike on a post on the home screen.
- **Undo** a post deletion to get the deleted post, along with all of its likes and comments, back.

A separate stack is kept for like/unlike undo, like/unlike redo, comment undo, comment redo, and post deletion undo.

Queue

A queue, implemented using a linked list, is used for the following purposes:

- Storing posts of the currently logged-in user.
- Enqueuing into the queue every time a new post is created.
- Deleting from the queue every time a post is deleted.

Linked List

A linked list is used for strong the record of the friends of the currently logged-in user. The following operations are performed:

- Adding a new node at the end of the linked list every time a new friend is added.
- Removing the corresponding node when a friend is removed.

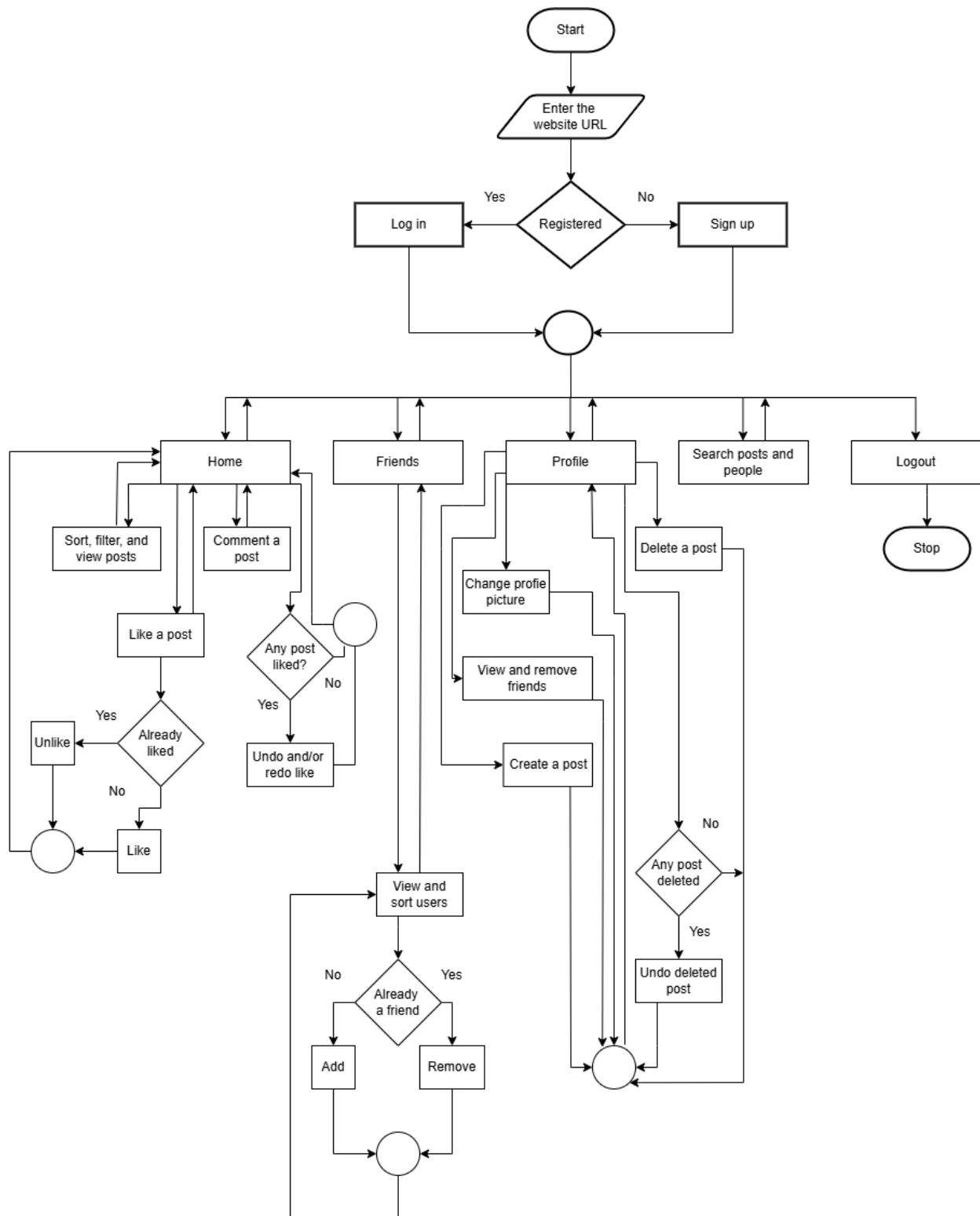
Sorting Algorithms

Algorithm	Usage
Insertion sort	Sorting posts by trend (through like count).
Quicksort	Sorting friends and users by their names in alphabetic and reverse-alphabetic order.
Merge sort	Sorting posts in terms of latest and oldest.

Database

Since the application is built in Django, it uses **SQLite** through Django's built-in ORM. Models include User, Post, Like, and Comment.

System Flow Diagram



Use Case: Posting and Image

Scope:

Cohesify

Level:

User goal

Primary Actor:

User

Stakeholders and Interests:

1. User: Wants to easily navigate to their profile, upload a picture, type in a caption, and post it such that it is displayed in the main feed of all other Cohesify users.
2. Other users: Want to view the posts uploaded by all users on the home screen.

Preconditions:

The user is logged in with a username and a password, and has the photo to be uploaded stored on their local machine.

Postconditions:

The user's new post is stored into the database, is visible to all the users (including the uploader) on home screen, and any user (including the uploader) can like, unlike, and comment on the post.

Main Success Scenario (Basic Workflow):

1. The user logs into Cohesify using their username and password.
2. They go to the profile screen.
3. Under the "Create a Post" title, they add a caption in the caption field.
4. They choose a picture from their device.
5. They click "Post".
6. The post is stored into the database, and its media is stored on the server.

7. The user is redirected back to the profile screen.
8. The newly created post is visible both on the profile screen (under “Your Posts”), and the home screen.

Extensions (Alternate Paths):

*a. At any time, the system fails:

1. The system logs the user out.
2. The user logs in again by entering their username and password.
3. The system redirects the user to the home screen.
4. If the user was previously providing any input, they provide it again.

2a. The user enters incorrect username and/or password:

1. The system redirects the user back to the login page, with the error message “Incorrect username/password.”
2. The user enters the correct username and password.

1a. The user enters incorrect username and/or password:

*Repeat 2a under *a.*

3a. The user does not enter a caption:

1. The system disallows the user to post until they enter a caption.
2. The user enters a caption.

4a. The user cannot find the picture they want to post on their device:

1. The system disallows the user to post until they choose a file that resides on their device.
2. The user chooses a file to post.

5a. Either the caption field, the file field, or both, are empty:

1. The system shows an error, highlighting the empty field(s) with an error message, asking the user to fill that field(s).

5b. The user uploads a very large-sized photo:

1. The server responds with a 413 Payload Too Large response code in the HTTP response.
2. The user clicks on the browser’s back button.
3. The user uploads a smaller-sized file.

6a. The database on the server has run out of physical storage to store the incoming file:

1. The server responds with a 507 Insufficient Storage response code in the HTTP response.
2. The user clicks on the browser's back button.
3. The user deletes some of their own posts.

3a. The user has no posts to delete:

1. The user waits for either the server to increase its physical storage capacity, or other users to delete their data.

7a. The user realizes they have uploaded the wrong post:

1. The user deletes the post from their profile screen.
2. *Repeat from step 3.*

1a. The user realizes they have deleted the wrong post:

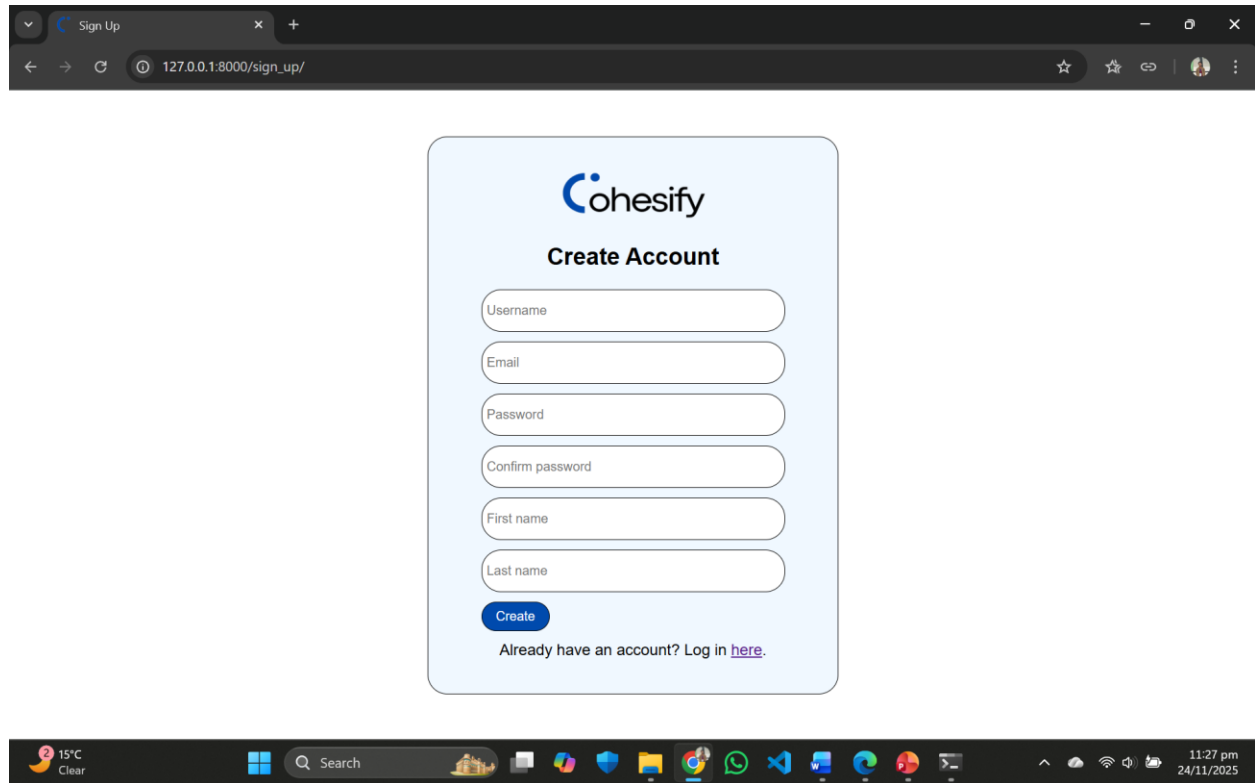
1. The user clicks on the "Undo delete" button.
2. The post, with all of its likes and comments, is restored in the database, and in the application's profile and home screens.

Frequency of Occurrence:

Could be regular.

Applications Screenshots

1. Create Account



The screenshot shows a web browser window with a single tab titled "Sign Up". The address bar displays "127.0.0.1:8000/sign_up/". The main content area features a light blue rounded rectangle containing the Cohesify logo at the top. Below the logo is the heading "Create Account". The form consists of seven input fields: "Username", "Email", "Password", "Confirm password", "First name", and "Last name". A blue "Create" button is positioned below the "Last name" field. At the bottom of the form, there is a link that reads "Already have an account? Log in [here](#)." The Windows taskbar is visible at the bottom of the screen, showing the system clock as 11:27 pm on 24/11/2025.

Figure 1: Create account screen

2. Login

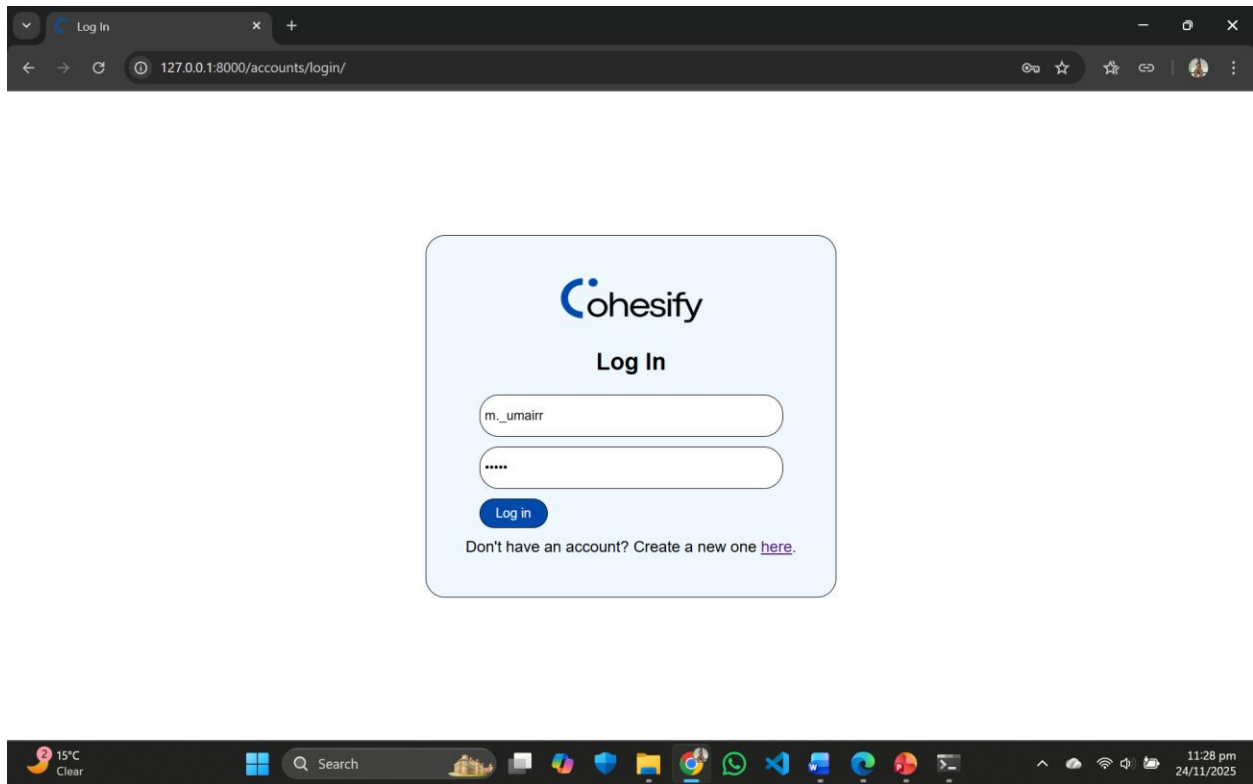


Figure 2: Login screen

3. Home

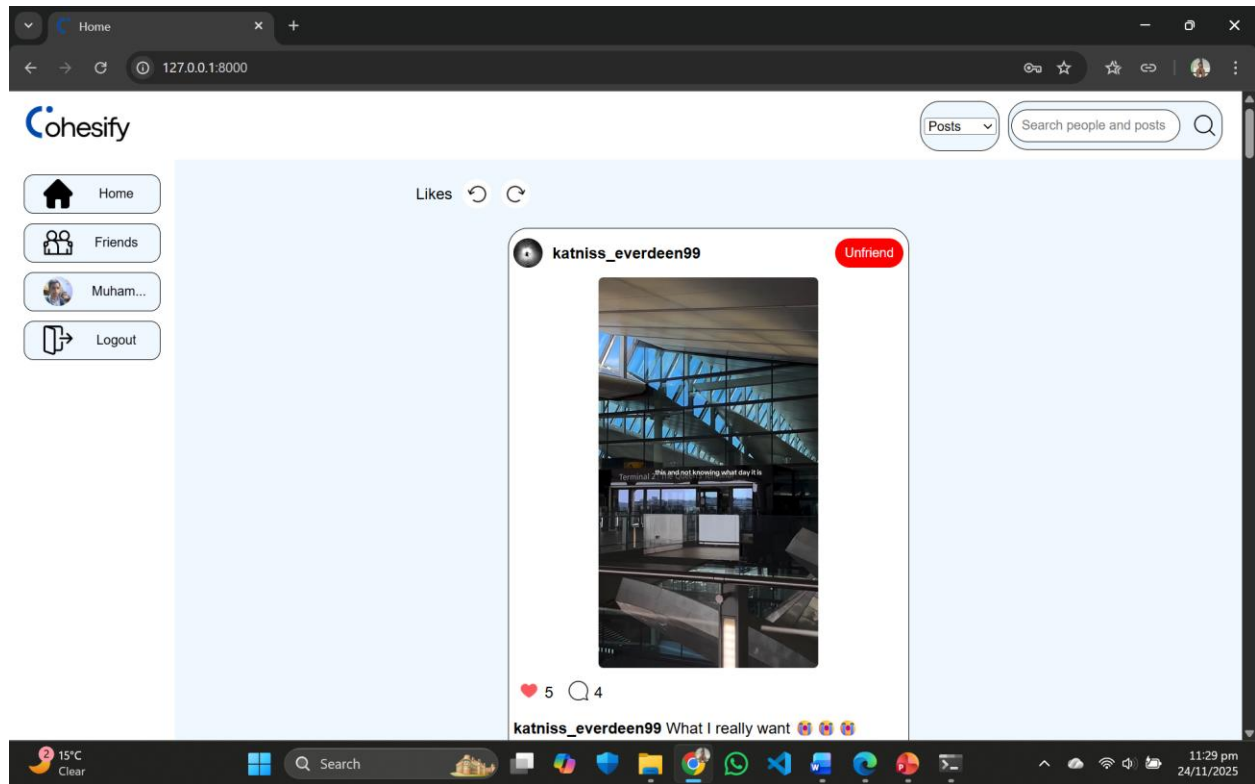


Figure 3: Home screen

4. Friends

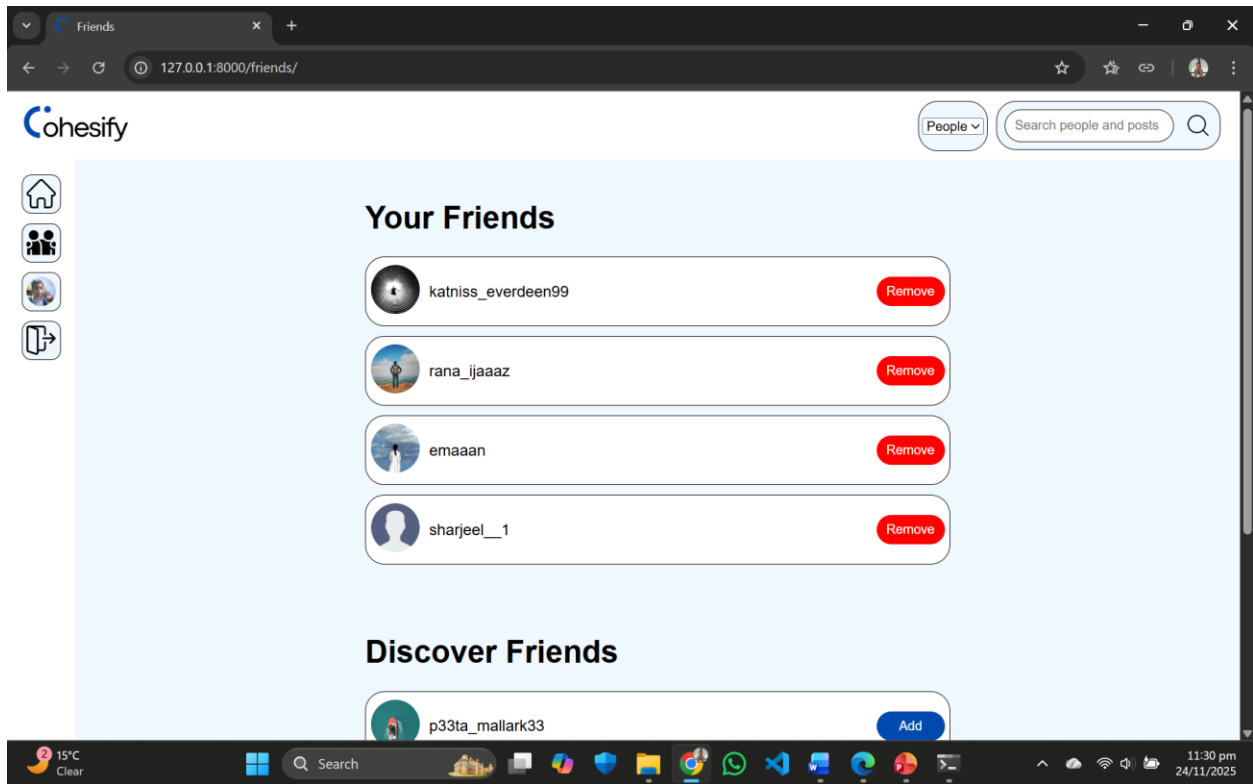


Figure 4: Friends screen

5. Profile

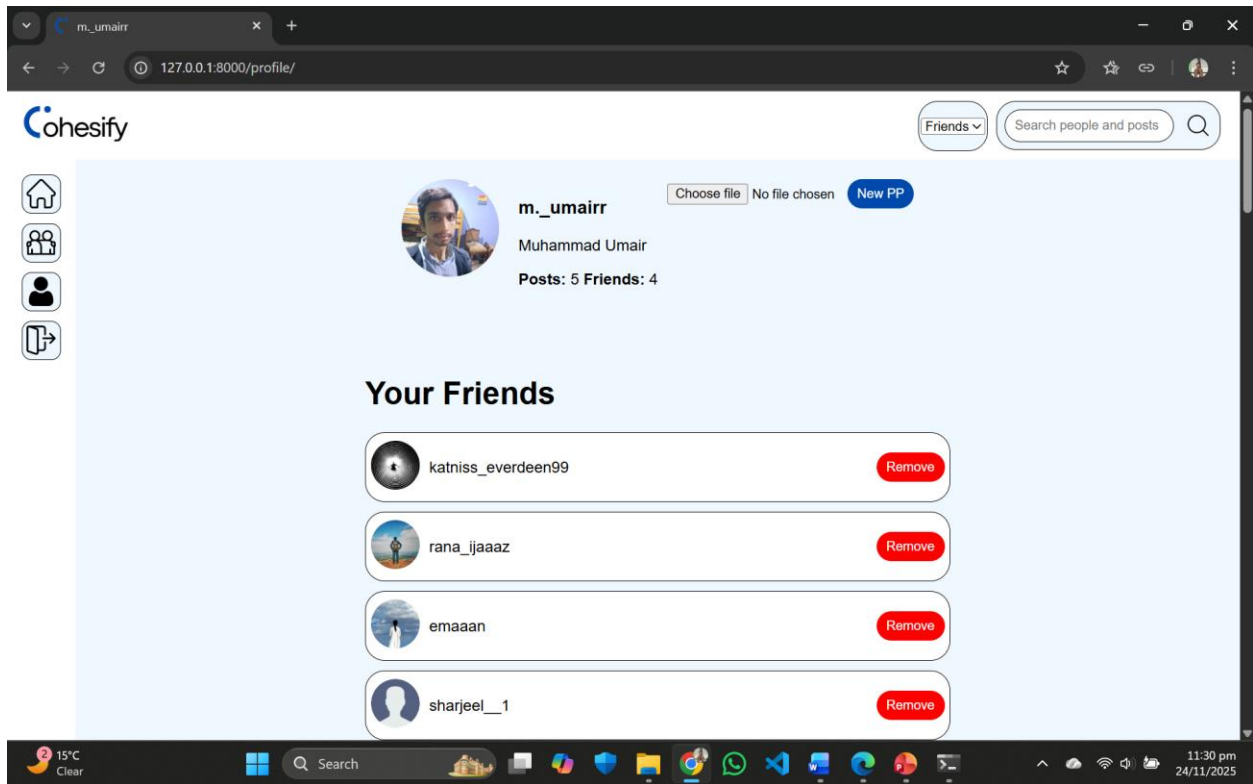


Figure 5: Profile screen (a)

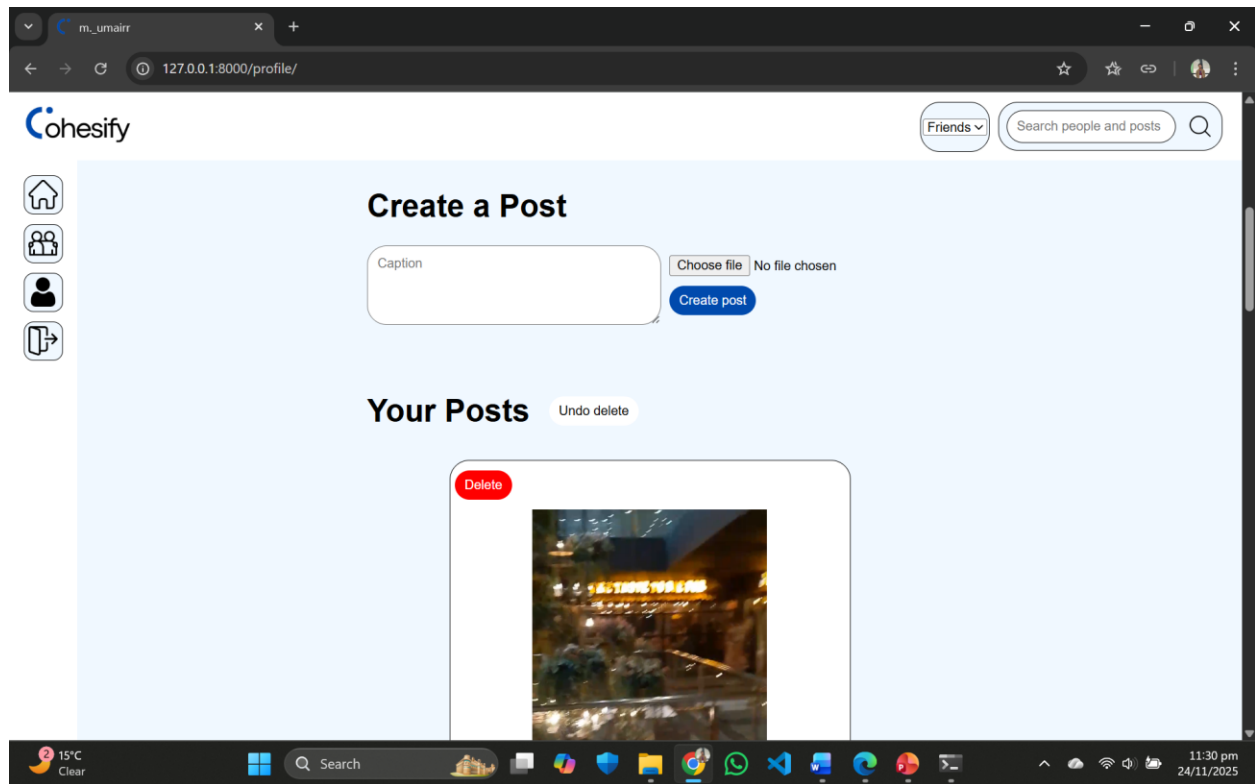


Figure 6: Profile screen (b)

6. Search

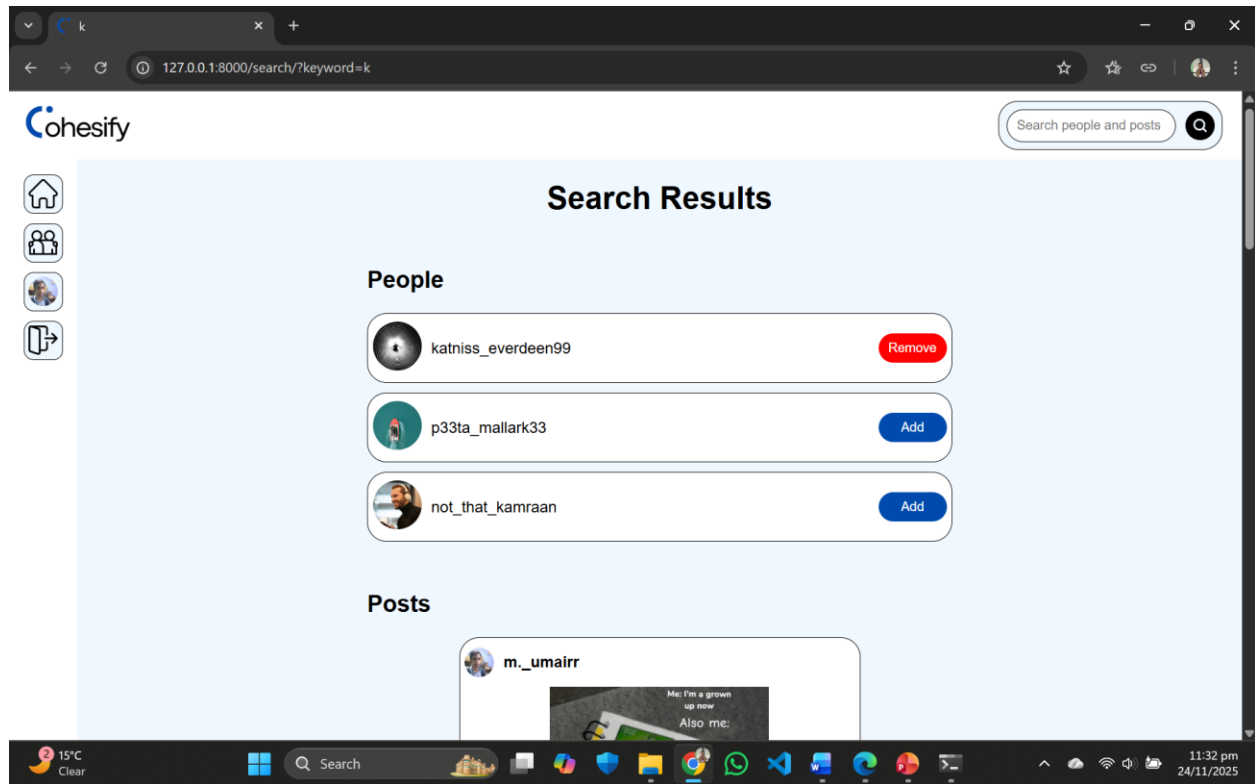


Figure 7: Search screen

7. Comments

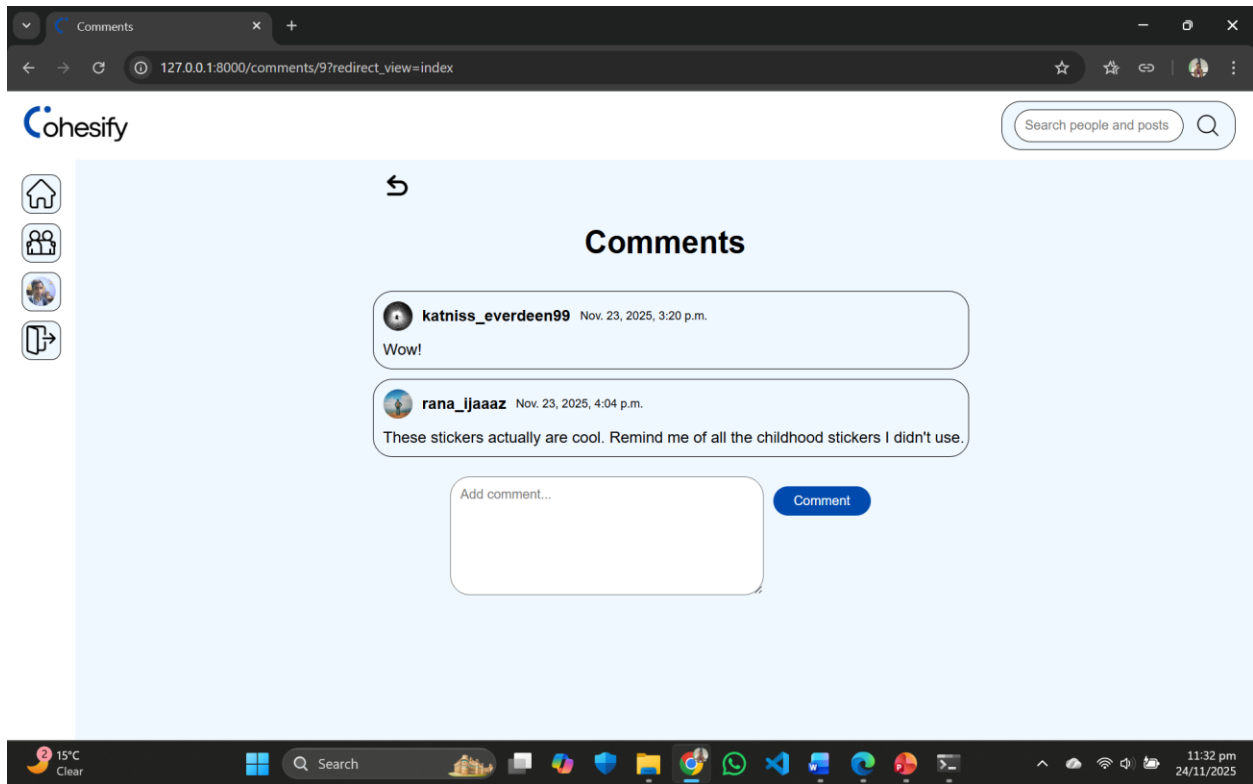


Figure 8: Comments screen

Performance Analysis

Operation	Data Structure/Algorithm	Time Complexity			Space Complexity
		Best	Avg.	Worst	
Sorting posts by trend	Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Sorting people by their names in alphabetic or reverse alphabetic order	Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Sorting posts in terms of latest and oldest	Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Storing a new post	Enqueue on a queue	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Deleting a post	Deletion in a queue	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Adding a new friend	Appending a linked list	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Removing a friend	Deleting from a linked list	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Undoing a like	Push, pop, and peek operations on a stack	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Redoing a like	Push, pop, and peek operations on a stack	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Undoing a post deletion	Popping from a stack	$O(1)$	$O(1)$	$O(1)$	$O(1)$